

Práctica 1: Uso de IA generativa

Autor: Jesús Muñoz Velasco

Sin duda el principal uso que se ha dado a la IA generativa¹ ha sido la corrección de errores en el código (en especial fallos lógicos). También la he usado para consultar cuáles serían las mejores estructuras de datos para ciertas variables de los algoritmos (abiertos, cerrados, etc). Quiero destacar las siguientes interacciones con la IA que quitando la corrección de fallos han sido las más interesantes:

```
ahora mi algoritmo se pasa de tiempo en A* (por bastante), qué puedo mejorar: package tracks.singlePlayer.evaluacion.src_MUNOZ_VELASCO_JESUS;
```

★ Ver razonamiento ▾

Tu A* tiene una base sólida, pero ahora mismo está sufriendo por lo que llamamos "**Muerte por mil New**" y un problema de **Heurística Ciega**.

El algoritmo se pasa de tiempo principalmente porque cada vez que exploras un sucesor, creas objetos pesados y clonas colecciones, sumado a que la heurística actual no guía al agente hacia la llave antes que a la meta.

Aquí tienes los puntos clave para que tu A* vuele:

- 1. El Gran Error: `isWallIfMove`**
En tu clase `Agente`, el método `isWallIfMove` crea un `new Node(n, move)`.
- El problema:** Ese constructor clona dos `HashSet`. Si tienes 4 movimientos posibles, estás creando y clonando 4 `HashSet` solo para mirar si hay un muro.
- Solución:** Comprueba las coordenadas directamente en el `mapaChar` usando `pos_x` y `pos_y` del padre sumando el movimiento. No crees un nodo hasta que estés seguro de que el movimiento es válido.

- 2. Heurística de dos etapas (Llave -> Meta)**
Tu heurística actual es la distancia Manhattan a la meta. El problema es que el A* intentará ir directo a la meta, se chocará porque no tiene la llave, y empezará a expandir nodos aleatoriamente hasta encontrar la llave por accidente.

Cambia `updateEstimatedCostTo` en `Node`:

¹En mi caso he usado Gemini con el modelo Pro.

Java

```
public void updateEstimatedCostTo(Node objetivo, Node posicionLlave) {
    if (!this.have_key && posicionLlave != null) {
        // Distancia a la llave + Distancia de la llave a la meta
        this.estimated_cost = distanceTo(posicionLlave) + posicionLlave.distanceTo(objetivo);
    } else {
        // Si ya tengo la llave, directo a la meta
        this.estimated_cost = distanceTo(objetivo);
    }
}
```

3. Simplifica el Estado (Hash/Equals)

En tu `hashCode` y `equals`, estás usando variables redundantes. Cuanto más grande sea el estado, más nodos "distintos" cree el A* y más memoria consume.

- `coins_left`, `keys_left` y `catapults_left` son redundantes si ya tienes los `HashSet` y la variable `coins`.
- **Limpia el `hashCode`**: Quédate solo con `pos_x`, `pos_y`, `coins`, `have_key`, `is_flying` y los dos `HashSet`.

4. Optimización de la lógica A*

Estás usando `mapaAbiertos` para evitar duplicados en la `PriorityQueue`. Es buena idea, pero puedes hacerlo más eficiente:

- En el bucle de sucesores, si un nodo ya está en `cerrados` con un coste menor, **ignóralo completamente**. No vuelvas a meterlo en `abiertos`. En juegos de este tipo, la primera vez que llegas a un estado suele ser la mejor (o casi).
- Evita `sucesor.isBetterThan(cerrados.get(sucesor))`. Si el estado es el mismo, pero el coste es mayor, descártalo inmediatamente.

5. ¿Cuántas monedas hay? (Tip Pro)

Si el número de monedas es pequeño (menos de 64), usar un `HashSet<Integer>` es muy lento debido al autoboxing y la clonación.

- **Alternativa:** Usa un `long` como **Bitmask**. Cada bit representa si una moneda ha sido recogida o no. Comparar un `long` en el `equals` es instantáneo comparado con comparar dos `HashSet`.

Sin duda lo más destacable de esta respuesta fue el punto 5 (el resto había cosas como la heurística que no podía cambiar por el enunciado de la práctica). Me sugirió usar un *bitmask* (máscara de bits) para optimizar la copia de los nodos (ya que antes tenía un *hashset* para almacenar las monedas y catapultas recogidas, y más tarde para las llaves también). Esto hacía muchísimo más eficiente todo. La siguiente consulta fue cómo implementar esto y cómo funcionaba (la parte de la implementación no la adjunto como imágenes porque es muy larga y es como ha quedado en mi código finalmente):

vale ya funciona increíblemente rápido. Necesito ahora que me expliques en profundidad cómo funciona la máscara y el mapa de bits y que me expliques el siguiente código: // Mapear cada objeto coleccionable a un bit del 0 al 63...



Ver razonamiento ▾

Me alegra mucho que ya vuele. Es la diferencia entre un algoritmo que "funciona" y uno que está **optimizado para competición**.

Lo que hemos hecho es pasar de una mentalidad de **objetos** (lenta y pesada) a una mentalidad de **bits** (la forma en que la CPU realmente piensa). Vamos a desmenuzarlo.

1. El concepto: ¿Qué es una Bitmask?

Imagina que tienes 64 bombillas en fila. Cada bombilla representa un objeto del mapa (una moneda o una catapulta).

- Si la bombilla está **apagada (0)**, el objeto sigue en el mapa.
- Si la bombilla está **encendida (1)**, el agente ya ha recogido ese objeto.

En Java, un `long` tiene exactamente **64 bits**. En lugar de tener un `HashSet` con objetos `Integer` (que ocupan mucha memoria y tardan en compararse), usamos un solo `long` como si fuera ese panel de 64 interruptores.

¿Por qué es tan rápido?

Para que el A* sepa si dos estados son iguales, debe comparar si ha recogido las mismas monedas.

- **Con `HashSet`**: Java tiene que iterar por los elementos, calcular hashes, comparar uno a uno...
- **Con Bitmask**: La CPU compara un `long` con otro en **un solo ciclo de reloj**. Es la operación más rápida que existe.

2. Explicación del código: El "Diccionario de Bits"

Este bloque de código es el **mapeo previo**. Se ejecuta una sola vez al principio para asignar a cada posición del mapa un "número de interruptor".

Java

```
// 1. Contador para saber qué bit toca asignar (del 0 al 63)
int indexCounter = 0;

// 2. Recorremos todo el mapa buscando cosas importantes
for (int x = 0; x < mapaChar.length; x++) {
    for (int y = 0; y < mapaChar[x].length; y++) {
        char c = mapaChar[x][y];

        // 3. ¿Es esta casilla un objeto coleccionable? (Monedas o Catapultas)
        if (c == 'c' || c == 'm' || c == '0' || c == '1' || c == '2' || c == '3') {

            // 4. Solo tenemos 64 bits en un 'long'. No podemos trackear más de 64 cosas
            if (indexCounter < 64) {
                // 5. Guardamos en un diccionario: "La posición (x,y) corresponde al bit indexCounter"
                // getPosKey convierte (x=5, y=2) en un único número como 50002 por ejemplo
                itemToIndex.put(getPosKey(x, y), indexCounter++);
            }
        }
    }
}
```

¿Qué acaba de pasar aquí?

Si el mapa tiene una moneda en (2,3) y otra en (10,5), el `HashMap` dirá:

- Posición 20003 → Bit 0
- Posición 10005 → Bit 1

3. ¿Cómo se usa esto después? (Operaciones Binarias)

Cuando tu agente pasa por la casilla (2,3), el código hace lo siguiente:

1. **Busca el índice:** `int idx = itemToIndex.get(20003);` (devuelve 0).
2. **Crea la máscara:** `long bit = (1L << idx);`
 - Esto hace un "desplazamiento a la izquierda". Si `idx` es 0, el bit es `...0001`. Si `idx` es 1, el bit es `...0010` (2^1).
3. **Marca como recogido (OR):** `coins_bitmask |= bit;`
 - El operador `|` (OR) enciende el interruptor correspondiente sin tocar los demás.
4. **Comprueba si ya se recogió (AND):** `(coins_bitmask & bit) == 0`
 - El operador `&` (AND) mira si el interruptor en esa posición específica está encendido. Si da 0, es que estaba apagado (moneda no recogida).

4. El impacto en el A*

La clave de todo es el método `hashCode()` y `equals()`.

Al usar la bitmask, el "Estado" de un nodo queda definido por:

`pos_x`, `pos_y`, `have_key`, `is_flying` y `coins_bitmask`.

Si el agente llega a la casilla (5,5) habiendo recogido la moneda A, y luego llega a (5,5) habiendo recogido la moneda B, el `coins_bitmask` será distinto. El A* entenderá correctamente que son **situaciones diferentes** y seguirá buscando. Pero si llega a (5,5) con la misma moneda recogida por un camino más largo, el `equals` detectará que es el **mismo estado** y lo descartará gracias a la tabla de `cerrados`.

¿Qué más podemos hacer?

Ahora que el motor de búsqueda es una bestia, el siguiente paso es la **Heurística de la Llave**. Sin ella, tu agente sigue siendo un "atleta ciego": corre muy rápido, pero no sabe que primero tiene que ir a por la llave antes de tocar la puerta.

¿Quieres que implementemos la lógica para que el `estimated_cost` cambie dinámicamente según si tienes la llave o no?

Ignorando la sugerencia acerca de la mejora de la heurística me resulto muy útil esta representación de "marcado" de posiciones visitadas para marcar cada objeto coleccionable. Más tarde tuve que mejorarla para implementar también las posiciones de las llaves que había recogido (ya que sino no me cuadraban las métricas con las de las soluciones administradas). Es la primera vez que uso una máscara de bits en un código propio (lo habíamos usado en FR pero para máscaras de redes) y me ha parecido una forma muy limpia y eficiente de representar esto (consigue que todos los nodos tengan una representación con tipos primitivos, sin estructuras complejas como *hashsets*).

Antes de esto también tengo consultas sobre qué estructuras eran las más eficientes para A* (tenía dos *hashmaps* para abiertos y cerrados respectivamente). Podría destacar la siguiente interacción (quizás la pregunta no tenía nada que ver, pero me sirvió la respuesta):

Un par de consejos para tu implementación:

1. **Cuidado con `equals`**: Veo que tienes un método `public boolean equals(Node node)` y otro `@Override public boolean equals(Object obj)`. Java siempre usará el segundo (el que recibe un `Object`) para las colecciones. Asegúrate de mantener el `@Override` para que `PriorityQueue` o `HashSet` funcionen correctamente.
2. **Rendimiento en A***: Si estás usando esto para un algoritmo de búsqueda de caminos, recuerda que `PriorityQueue.contains(Object)` es lento ($O(n)$). Es mejor combinar la `PriorityQueue` con un `HashSet<Node>` (cerrados) para que las comprobaciones de "si ya visité este estado" sean instantáneas ($O(1)$).
3. **Heurística**: En tu método `inspect()`, mencionas que el coste estimado son las monedas. Asegúrate de que tu heurística sea **admisibile** (que nunca sobreestime el coste real) para que la `PriorityQueue` te devuelva siempre el camino óptimo.

Podemos ver que en el punto 2 nos sugiere usar una estructura paralela (resultado que acabé implementando en mi práctica) para la cola con prioridad.

Sin duda he usado la IA generativa para muchas más cuestiones, pero han sido sobre todo corrección de errores (ya arreglados en mi código final y sin mayor relevancia). Estas interacciones anteriores son las que más han guiado el desarrollo de mi práctica en cuanto al uso de la IA.